Leaf queries cannot fetch multiple `query` clauses. For example, they are not designed to search for movies that match a title but must *not* match a particular actor *and* have been released during a specific year *and* have a rating that is not below a certain number. The advanced requirement of logically combining certain clauses to serve a complex query is not possible with a leaf query, which leads to the introduction of compound queries.

Compound queries allow us to create complex queries by combining leaf queries and even other compound queries using logical operators. A Boolean (`bool`) query, for example, is a popular compound query that supports clauses like `must`, `must_not`, `should`, and `filter`. We can write significantly complex queries using compound queries, like this example.

Listing 8.16 Compound query

```
GET movies/_search
{
  "query": {
    "bool": {
      "must": [{"match": {"title": "Godfather"}}],
      "must_not": [{"range": {"rating": {"lt": 9.0}}}],
      "should": [{"match": {"actors": "Pacino"}}],
      "filter": [{"match": {"actors": "Brando"}}]
    }
  }
}
```

This compound query combines a handful of leaf queries joined by logical operators. It fetches all movies that *must* match the title *Godfather, and must not* have a rating less than 9. The query *should* also consider movies with the actor Pacino. Finally, it filters out everything except movies with the actor Brando. That's a lot! But fear not; we work with advanced querying using compound queries in upcoming chapters.

Leaf queries (as well as advanced queries) are wrapped in the `query` object of the search request. Other than implementing the advanced query's logic (which may sometimes be too complex), we should see no significant difference when writing compound queries.

There are several crosscutting features, such as sorting, pagination, and highlighting, that any type of a search query can use. They are not specific to `term` or `match`-level queries, compound queries, or leaf queries. We discuss these features in detail in the following section. We also use these features now and then during search and aggregation.

8.8 Search features

Elasticsearch provides capabilities to add features to queries and results. We can manipulate source documents in the response by asking Elasticsearch to return a full document or only specific fields. We can sort documents based on one or more fields in addition to sorting on the document's relevancy score. Elasticsearch lets us paginate